

Module 3

Uninformed Search

Nirmala Shinde Baloorkar

Assistant Professor

Department of Computer Engineering

Outline

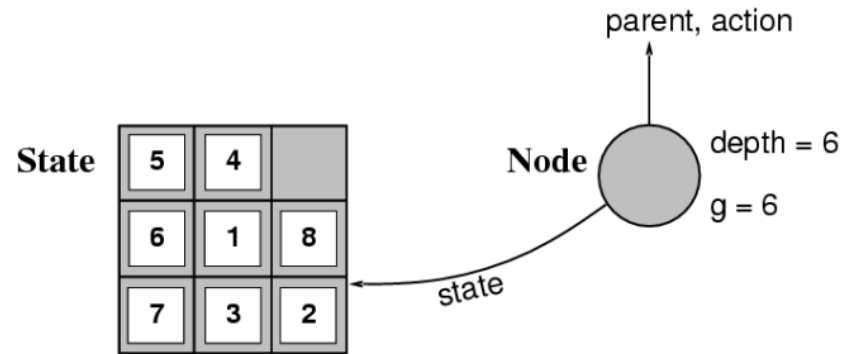
- Blind search strategies
 - Breadth-first search
 - Depth-first search
 - Depth-limited search
 - Iterative deepening search
- For each of these strategies
 - The algorithm
 - The time and space complexities
 - When to select a particular strategy

Uninformed (“Blind”) Search Strategies

- No information is available other than
 - The current state
 - ♦ Its parent (perhaps complete path from initial state)
 - ♦ Its operators (to produce successors)
 - The goal test
 - The current path cost (cost from start state to current state)

Terms

- **Node:** Represents a state in the problem along with metadata (e.g., path cost, parent, action).

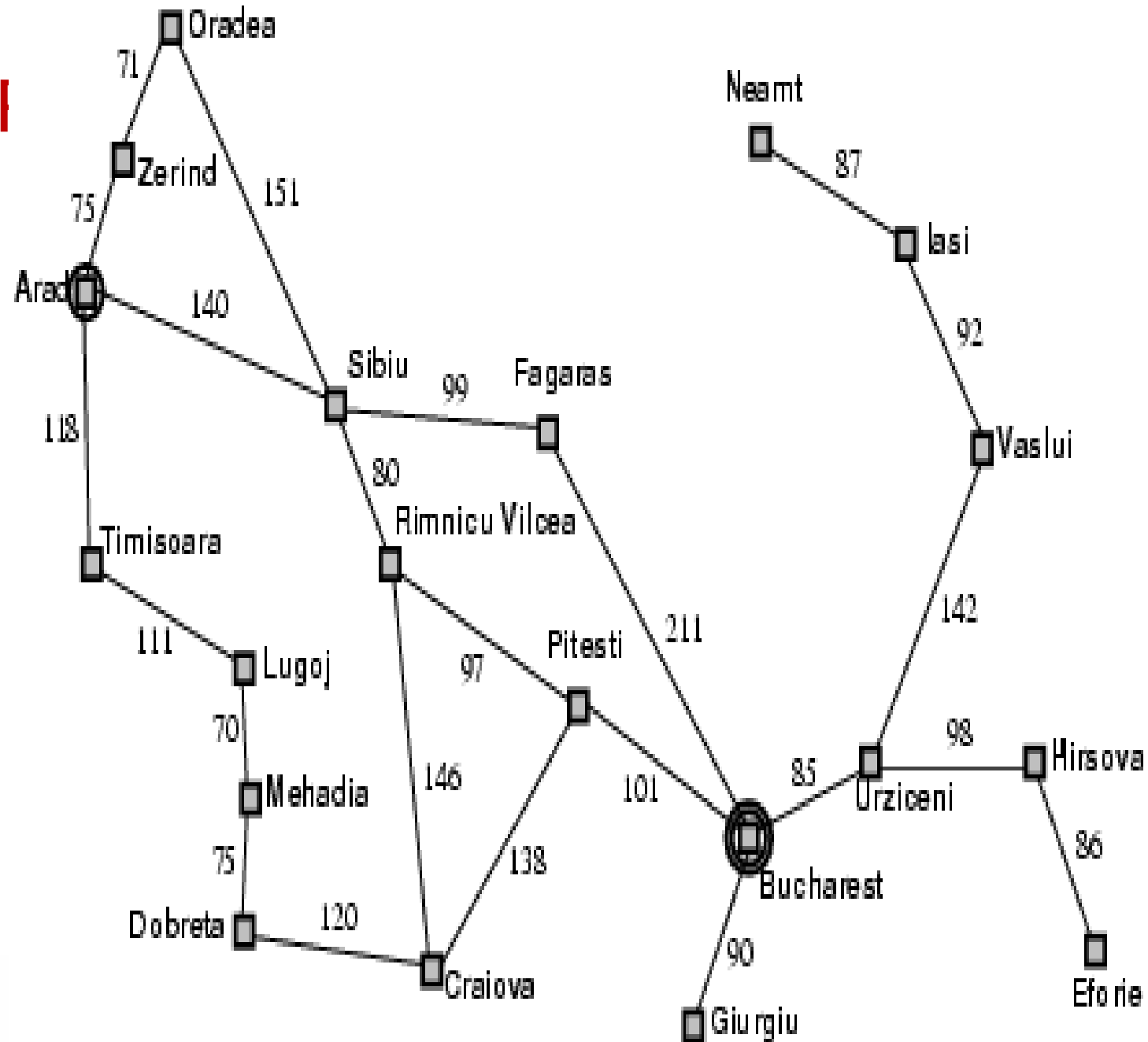


- **Expansion:** Generating successors from a node using valid actions.
- **Fringe:** The set of nodes waiting to be explored.

Handling repeated states

- Initialize the **fringe** using the **starting/initial state**
- While the fringe is not empty
 - Choose a fringe node according to **search strategy** and take it off the fringe
 - If the node contains the **goal state**, return solution
 - Else **expand** the node and add its children to the fringe
- To handle repeated states:
 - Every time you expand a node, add that state to the **explored set**; do not put explored states on the fringe again
 - Every time you add a node to the fringe, check whether it already exists in the fringe with a higher path cost, and if yes, replace that node with the new one

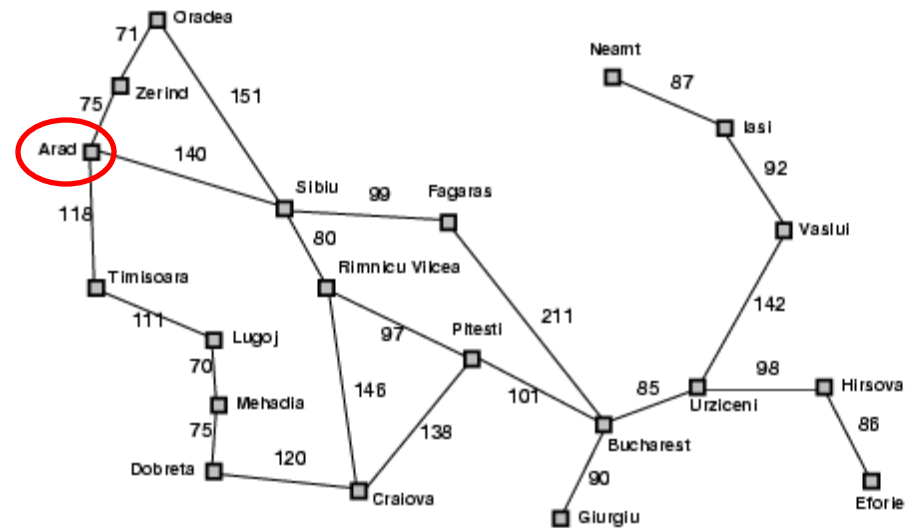
Example: I



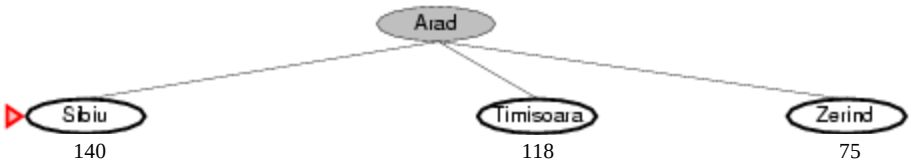
Search without repeated states



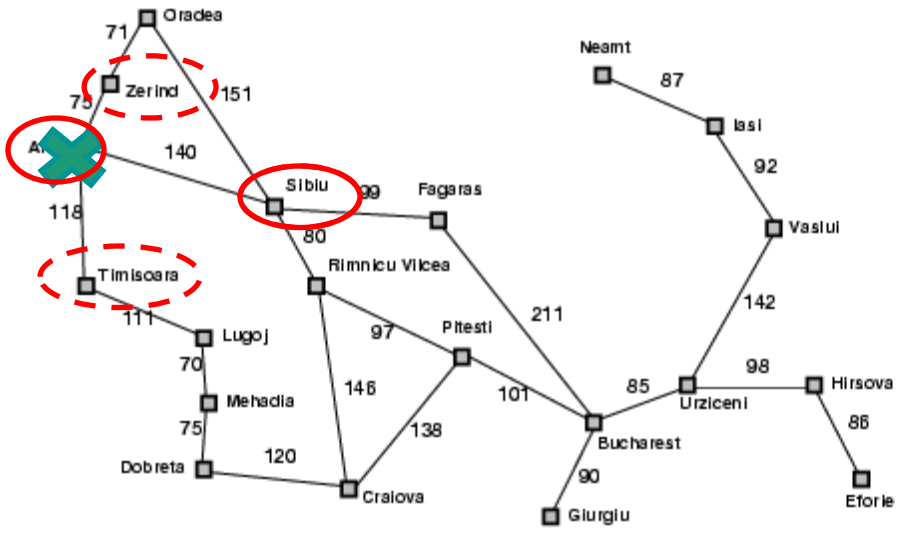
Start: Arad
Goal: Bucharest



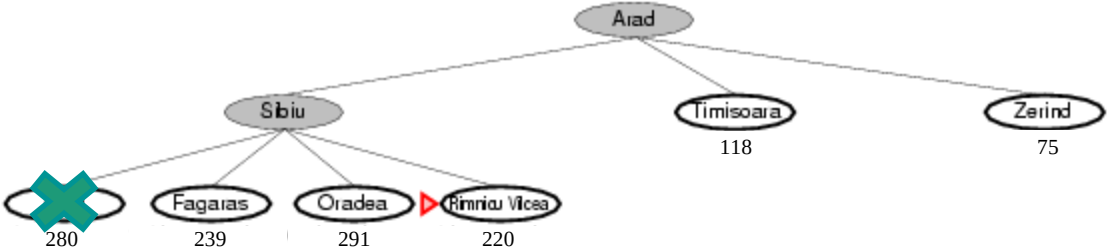
Search without repeated states



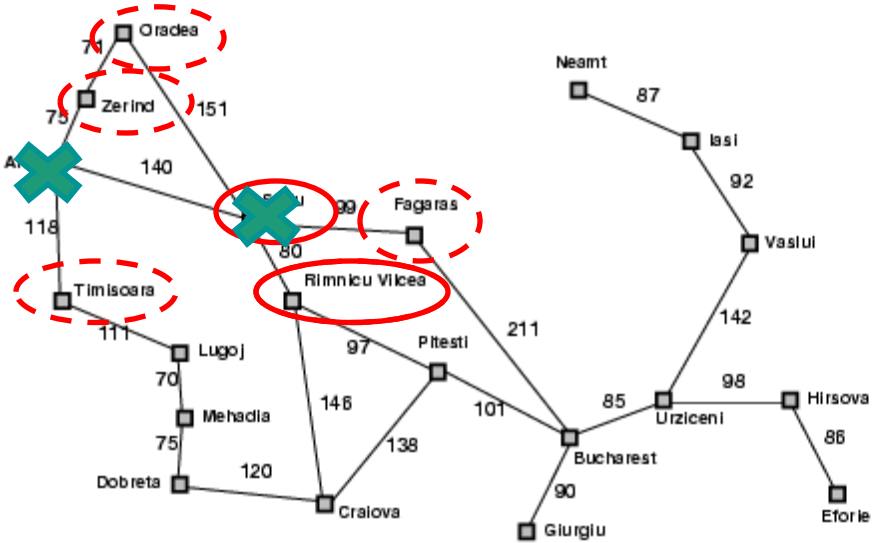
Start: Arad
Goal: Bucharest



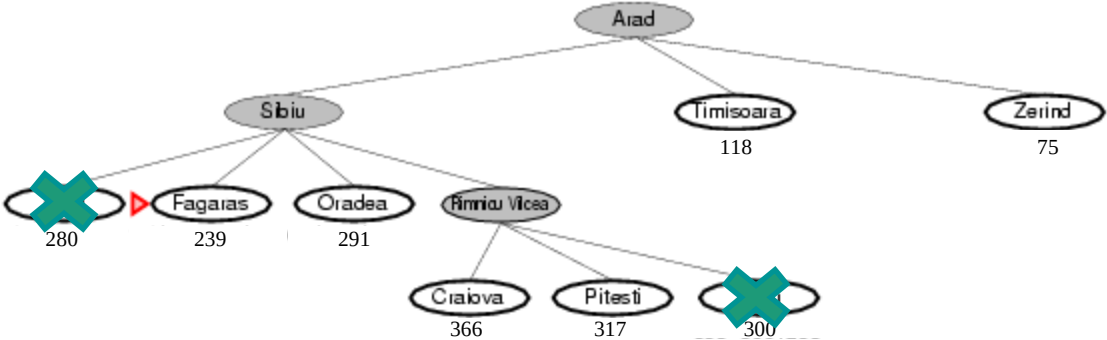
Search without repeated states



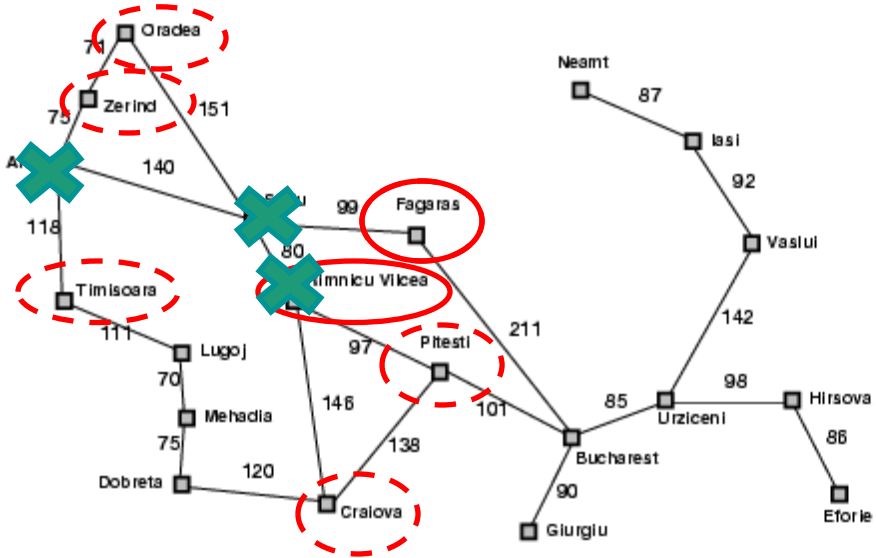
Start: Arad
Goal: Bucharest



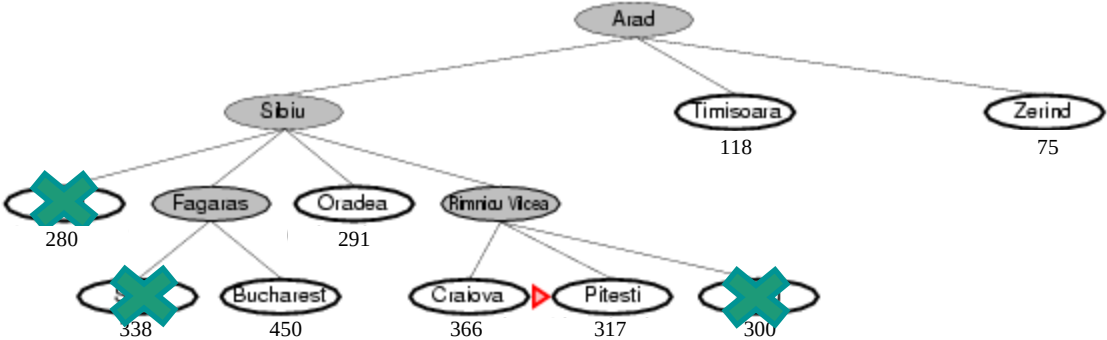
Search without repeated states



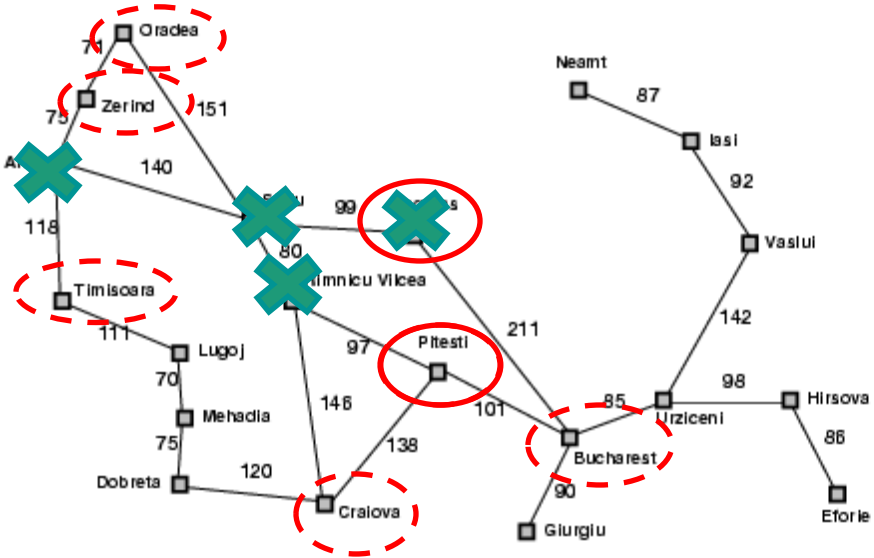
Start: Arad
Goal: Bucharest



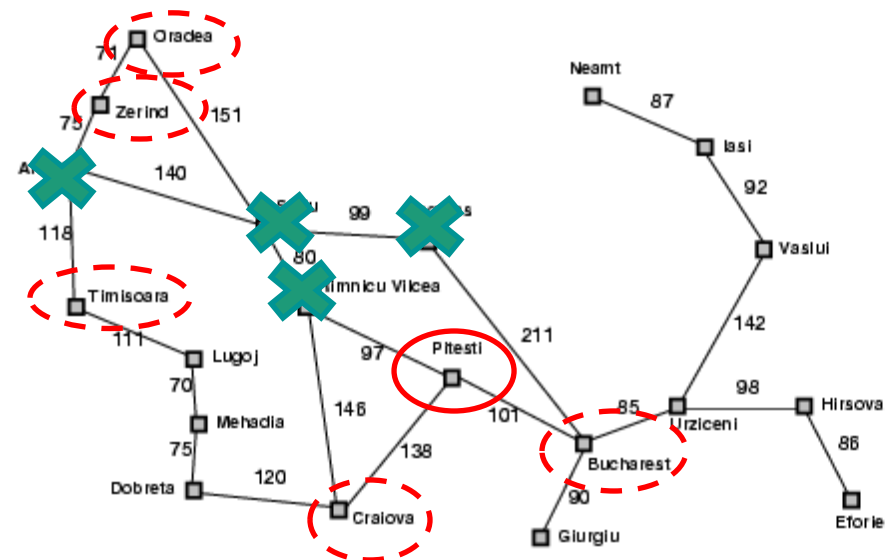
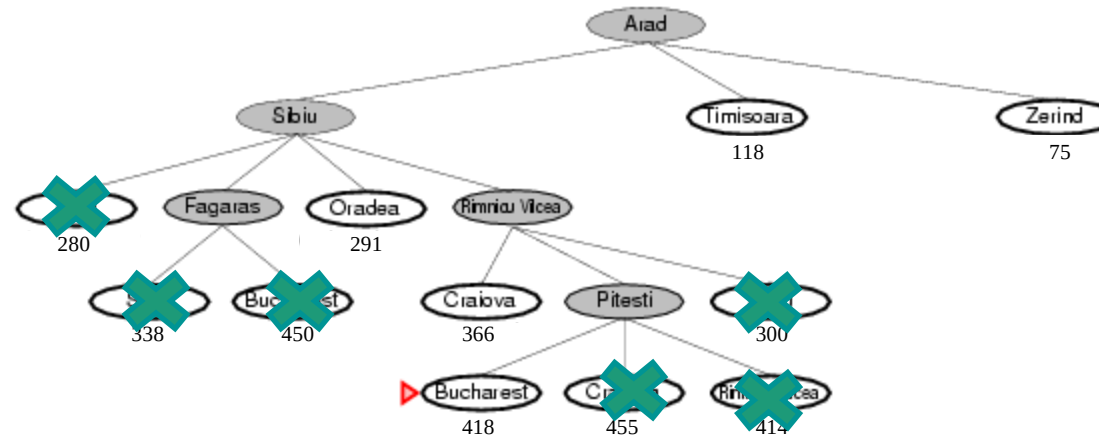
Search without repeated states



Start: Arad
Goal: Bucharest

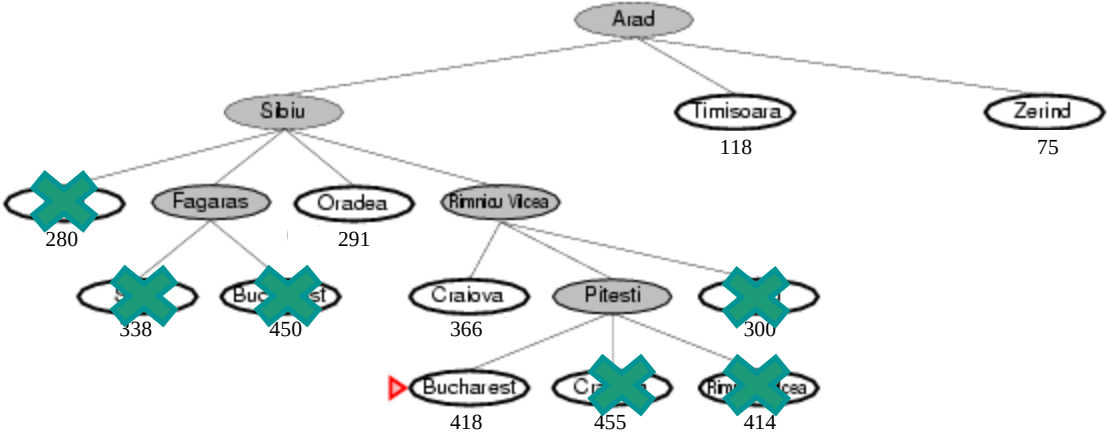


Search without repeated states

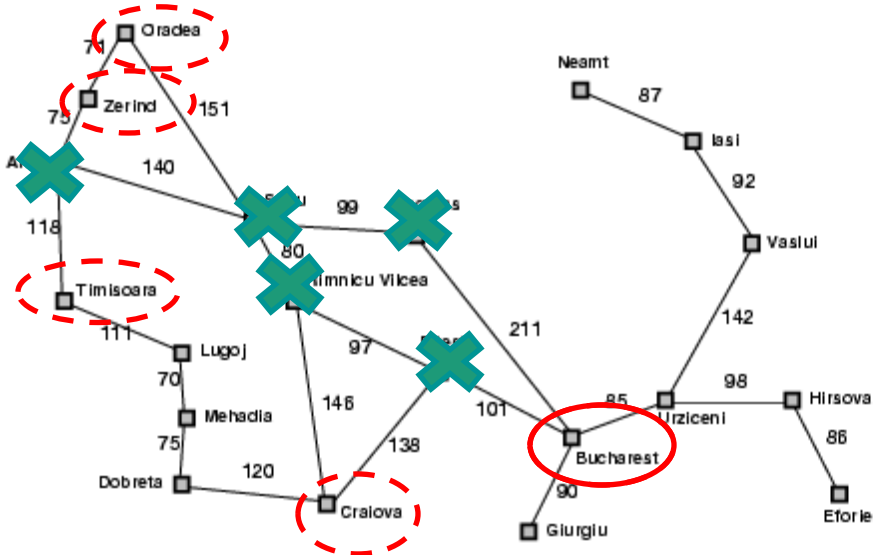


Start: Arad
Goal: Bucharest

Search without repeated states



Start: Arad
Goal: Bucharest



Breadth First Search

Algorithm

Let *fringe* be a list containing the initial state

loop do

if *fringe* **is empty** **then return** failure

 Node $\leftarrow$ remove-first(*fringe*)

if Node is goal state **then**

return the corresponding solution

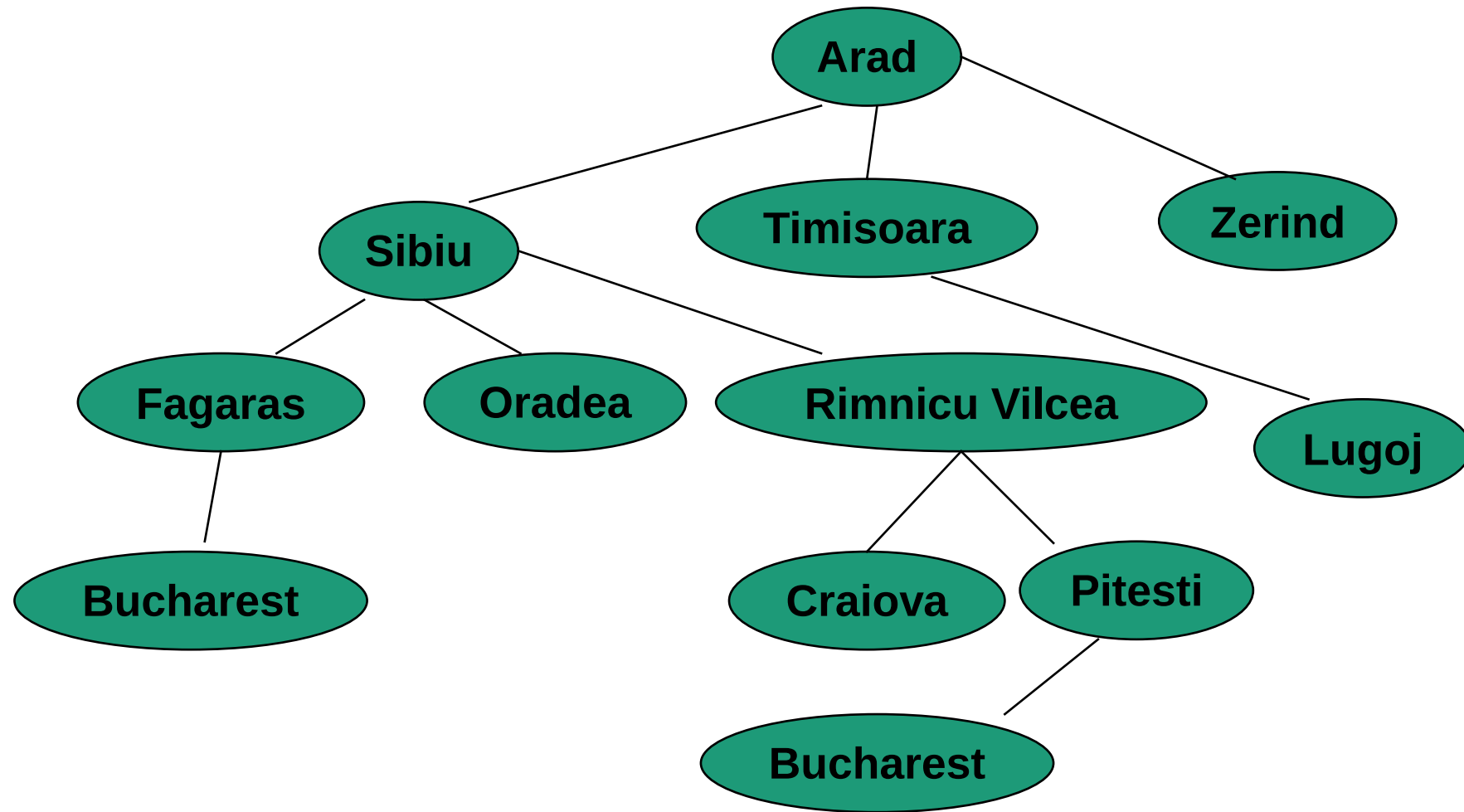
else expand all successor of Node and

 add the resulting(newly generated) nodes to the Back of
 fringe

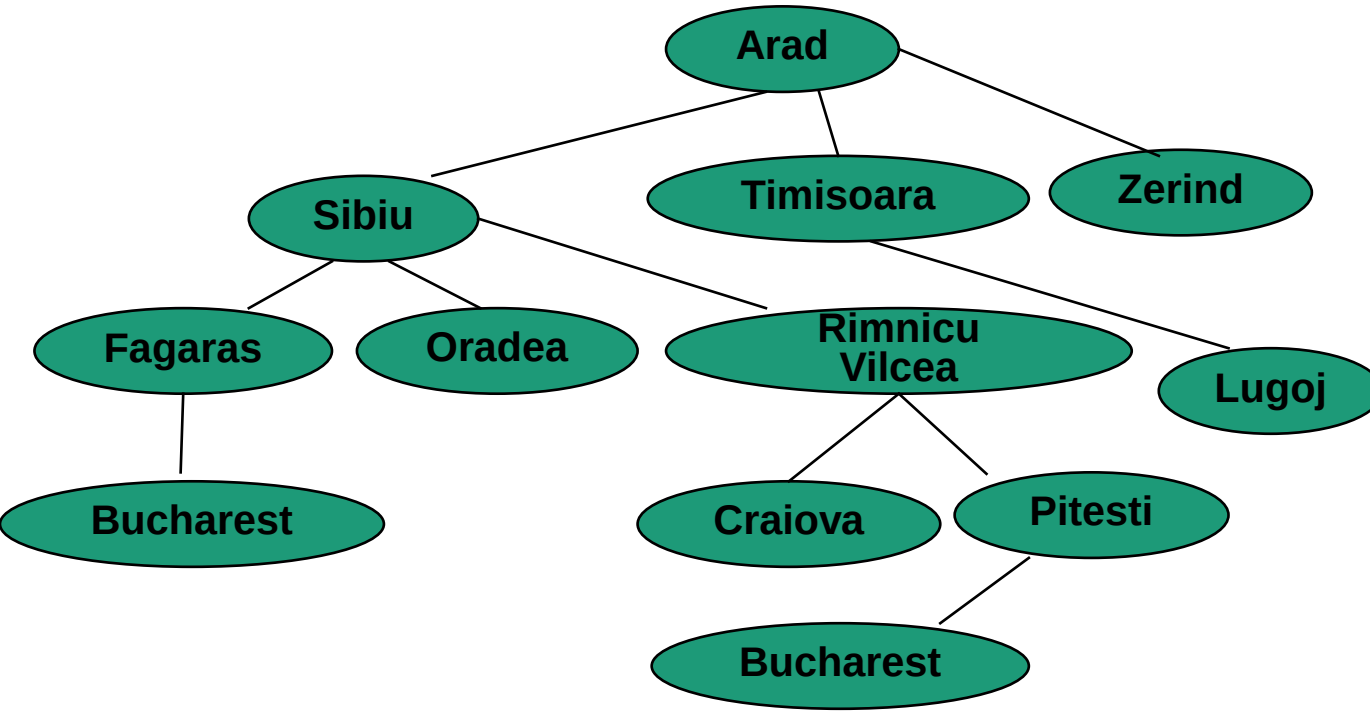
end

Node will be expanded in a FIFO order.

Example – Search Tree



BFS



Fringe

Closed

(A)

Nil

(S T Z)

(A)

(T Z F O RV)

(A S)

(Z F O RV L)

(A S T)

(F O RV L)

(A S T Z)

(O RV L B)

(A S T Z F)

(RV L B)

(A S T Z F O)

(L B C P)

(A S T Z F O RV)

(B C P)

(A S T Z F O RV L)

(C P)

Goal Node Found

Path : Arad-Sibiu-Timisoara-Zerind-Fagaras-Oradea-Rimnicu Vilcea-Lugoj-Bucharest

Breadth-First Search

- Complete?
- Optimal?
- Time complexity?
- Space complexity?

Yes

Yes

Exponential: $O(b^d)$

Exponential: $O(b^d)$

In practice, the memory requirements are typically worse than the time requirements

Breadth First Search

- Advantages
 - Finds the path of minimal length to the goal.
- Disadvantages
 - Requires the generation and storage of a tree whose size is exponential the depth of the shallowest goal node

Depth First Search

Algorithm

Let *fringe* be a list containing the initial state

loop do

if *fringe* is empty **then return** failure

 Node $\leftarrow$ remove-first(*fringe*)

if Node is goal state **then**

return the corresponding solution

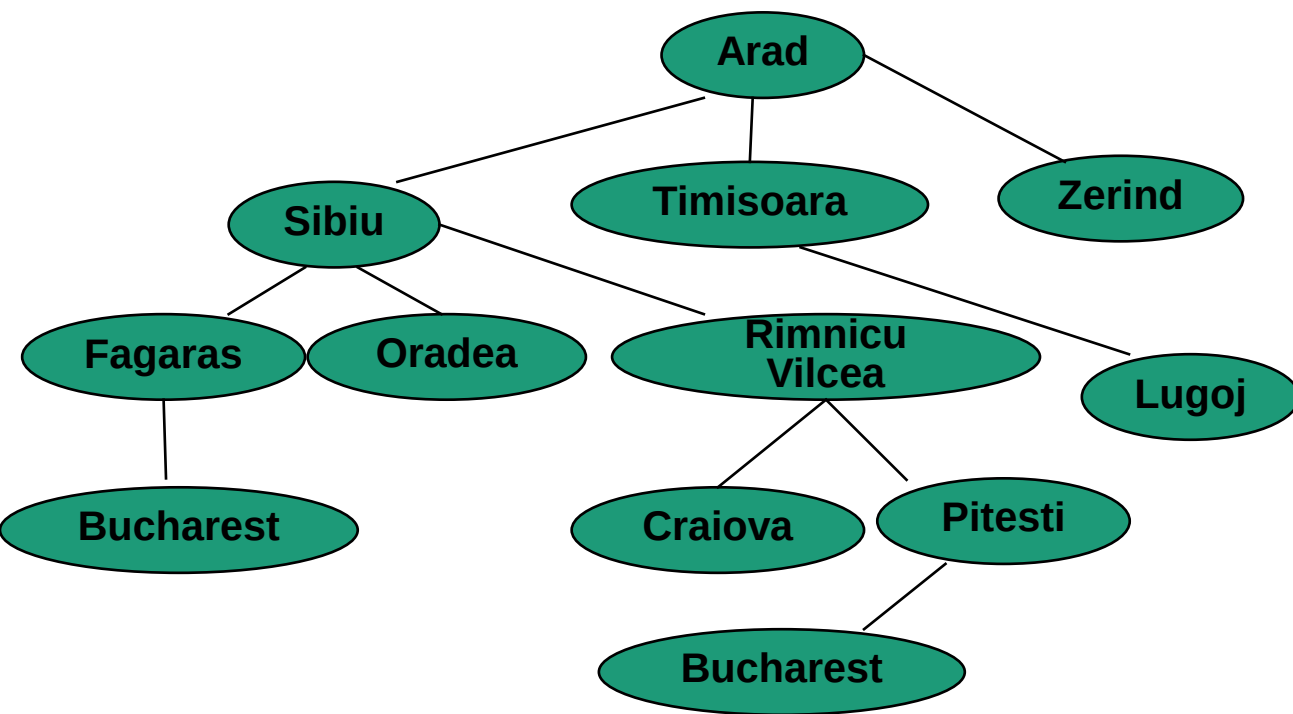
else expand all successor of Node and

 add the resulting(newly generated) nodes to the front of
 fringe

end

Node will be expanded in a LIFO order.

DFS



Fringe

(A)

(Z T S)

(Z T RV O F)

(Z T RV O B)

(Z T RV O)

Closed

Nil

(A)

(A S)

(A S F)

Goal Node Found

Path : Arad-Sibiu-Fagaras-Bucharest

Properties of depth-first search

- **Complete?**

Fails in infinite-depth spaces, spaces with loops

Modify to avoid repeated states along path

□ complete in finite spaces

- **Optimal?**

No – returns the first solution it finds

- **Time?**

Could be the time to reach a solution at maximum depth m : $O(b^m)$

Terrible if m is much larger than d

But if there are lots of solutions, may be much faster than BFS

- **Space?**

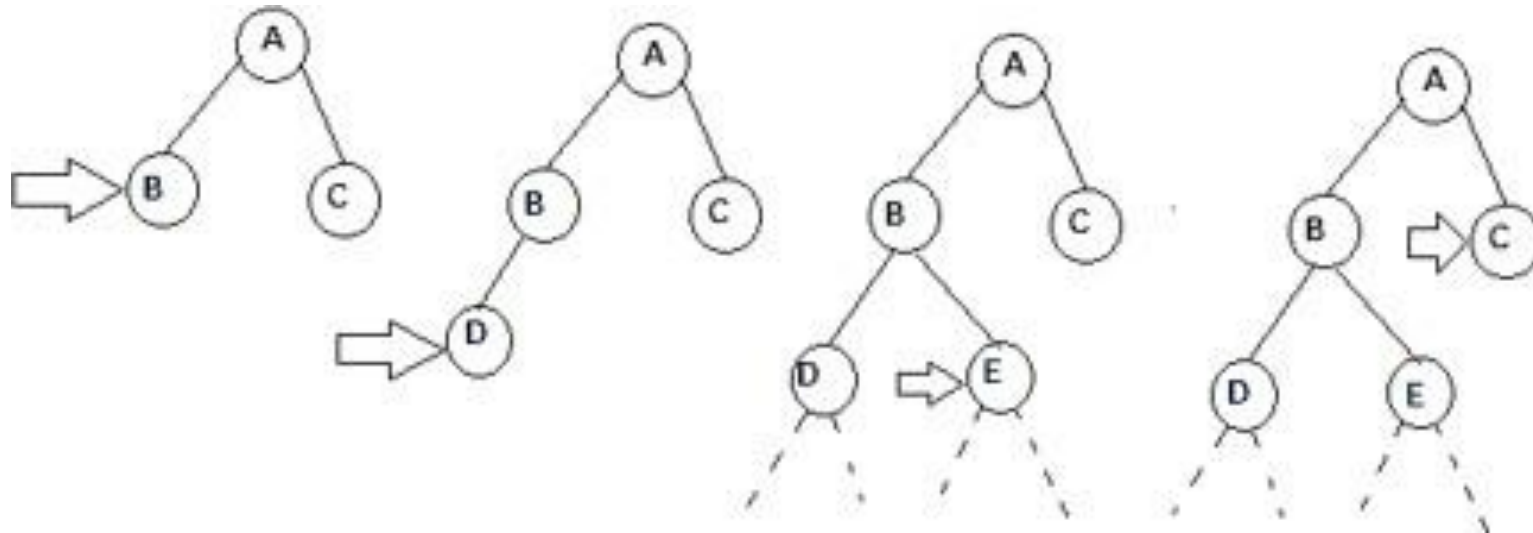
$O(bm)$, i.e., linear space!

Depth Limited Search (DLS)

- Depth-limited search is a variant of depth-first search designed to handle infinite state spaces by imposing a predetermined depth limit. In this approach, nodes at the specified depth are treated as if they have no successors.
- **Termination Conditions**
 - A solution is found within the depth limit.
 - No solution exists within the given depth limit.

Depth Limited Search (DLS)

- If depth is fixed to 2, DLS carries out depth first search till second level in the search tree.



Depth Limited Search (DLS)

Let *fringe* be a list containing the initial state with depth = 0

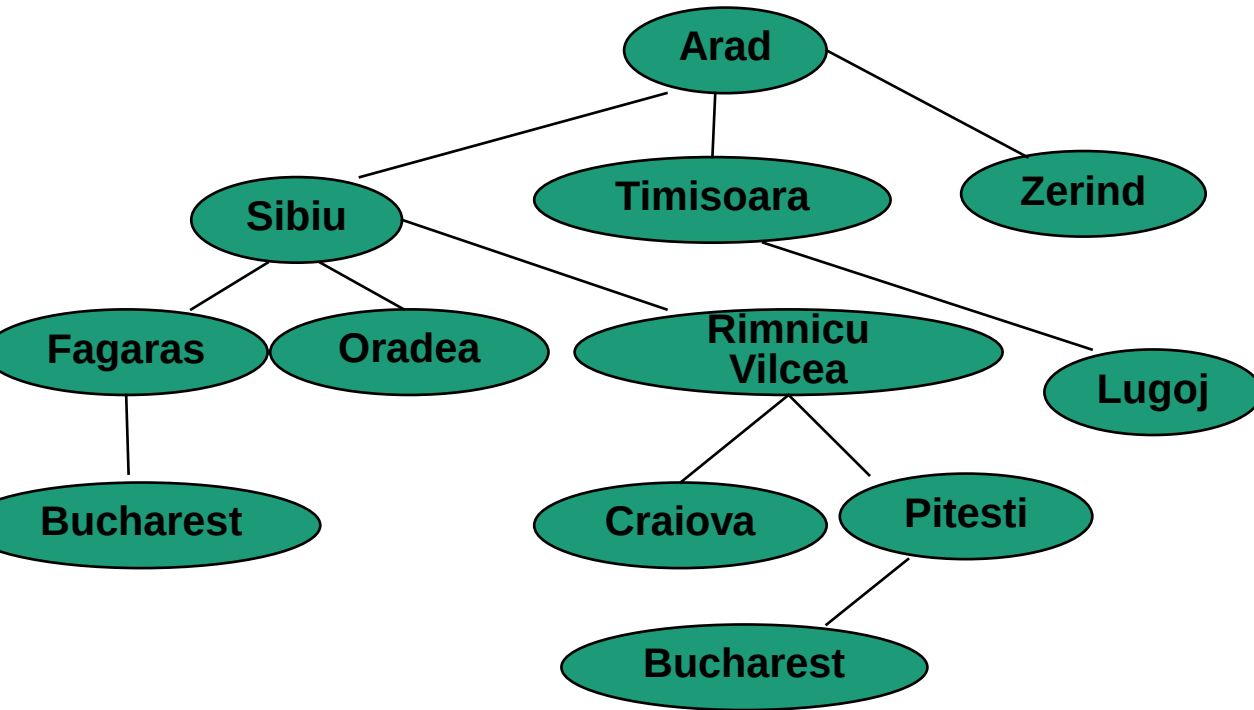
Algorithm

```
loop do
  if fringe is empty then return failure
  Node ← remove-first(fringe)
  if Node is goal state then
    return the corresponding solution
  else if depth of node < depth limit then
    expand all successor of Node and
    add the resulting(newly generated) nodes to the front of
    fringe with incremented depth
end
```

Node will be expanded in a LIFO order.

DLS

Depth Limit : 2



Fringe

(A) (Expand d=0)

(Z T S)

(Z T RV O F)

(Z T RV O)

(Z T RV O)

(Z T)

(Z)

()

Closed

Nil

(A)

(A S)

(A S F)

(A S F O)

(A S F O RV)

(A S F O RV T)

(A S F O RV T Z)

Goal Not Found

Path : NOT Found

DLS

- **Advantages:**

- Depth-limited search is Memory efficient.

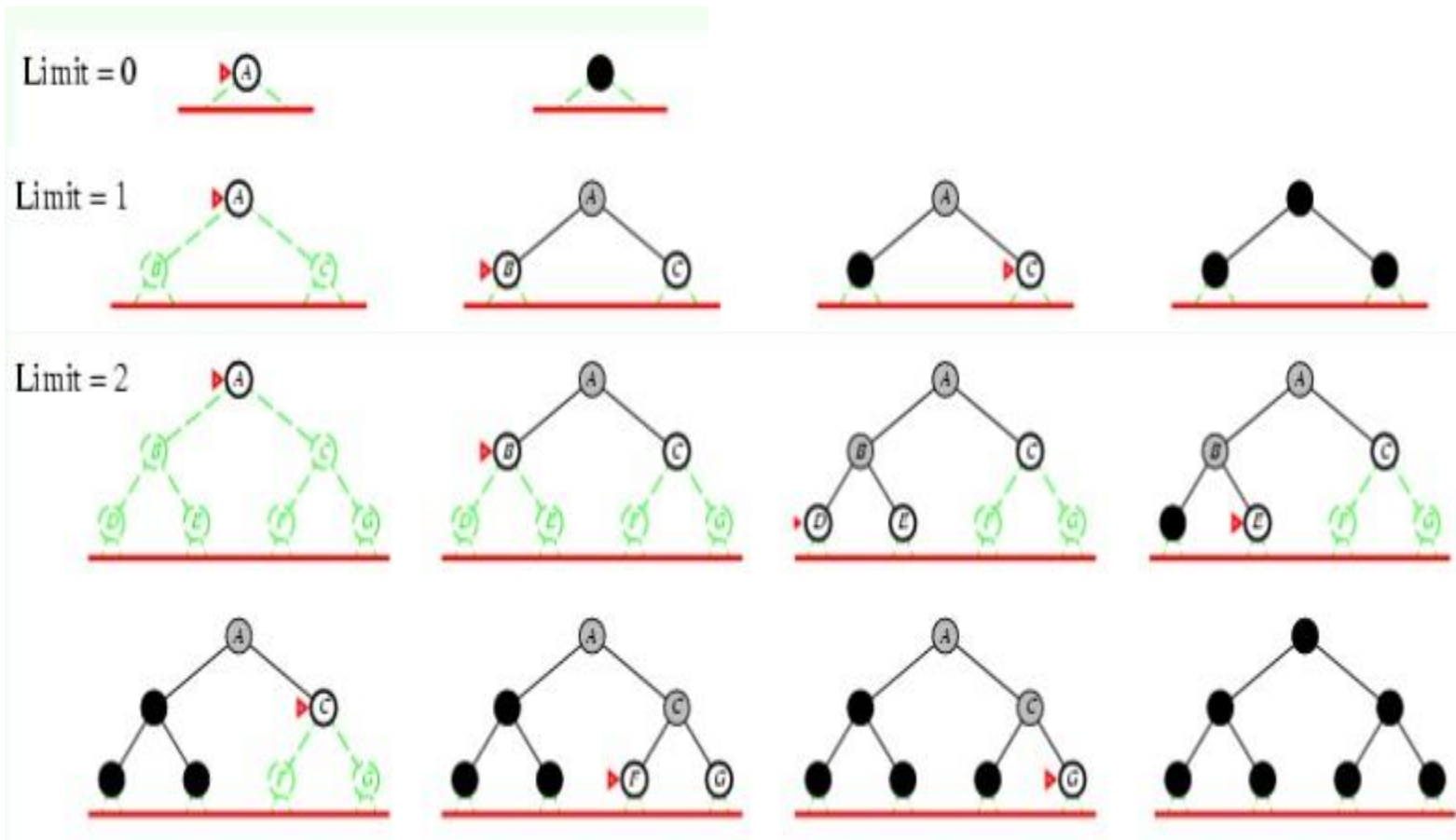
- **Disadvantages:**

- Depth-limited search also has a disadvantage of incompleteness.
- It may not be optimal if the problem has more than one solution.

Iterative Deepening Search (IDS)

- Iterative deepening search is a depth-first tree search that determines the optimal depth limit by incrementally increasing the limit.
 - It begins with a depth limit of **0** and gradually increments it to **1**, **2**, and so forth, until a goal is found.
 - The process concludes when the depth limit equals **d**, the depth of the shallowest goal node.
 - IDS effectively combines the advantages of both depth-first search and breadth-first search.

Example - IDS



IDS

- **Advantages:**

- It combines the benefits of BFS and DFS search algorithm in terms of fast search and memory efficiency.

- **Disadvantages:**

- The main drawback of IDDFS is that it repeats all the work of the previous phase.

Problem

2		3
1	8	4
7	6	5

Initial State

1	2	3
8		4
7	6	5

Goal State

Goal formulation

Arrange 1-8 numbered tiles as per the sequence given in problem statement

Problem formulation

- State description- 3 X 3 grid of tiles numbered through 1 to 8 and a blank tile.
- Initial state – given combination of 3 X 3 grid
 - 8-puzzle(2,\$,3,1,8,4,7,6,5)
- Actions- left(), right(), up(), down()
- Successor function (on goal state)
 - <left(), 8-puzzle(1,2,3,\$,8,4,7,6,5)>
 - <right(), 8-puzzle(1,2,3,8,4,\$,7,6,5)>
 - <up(), 8-puzzle(1,\$,3,8,2,4,7,6,5)>
 - <down(), 8-puzzle(1,2,3,8,6,4,7,\$,5)>

2		3
1	8	4
7	6	5

IS

1	2	3
8		4
7	6	5

GS

2		3
1	8	4
7	6	5

A

left

right

down

	2	3
1	8	4
7	6	5

B

2	3	
1	8	4
7	6	5

C

2	8	3
1		4
7	6	5

D

down

down

left

right

down

1	2	3
	8	4
7	6	5

E

2	3	4
1	8	
7	6	5

F

2	8	3
	1	4
7	6	5

G

2	8	3
1	4	
7	6	5

H

2	8	3
1	6	4
7		5

I

right
~~left~~

down

left

down

up

down

up

down

1	2	3
8		4
7	6	5

J

1	2	3
7	8	4
	6	5

K

2	3	4
1		8
7	6	5

L

2	3	4
1	8	5
7	6	

M

N

O

P

Q

left

R

BFS, DFS, DLS, IDS

- BFS
- DFS
- DLS
- IDS

Puzzle

- Initial State

	2	3
1	8	4
7	6	5

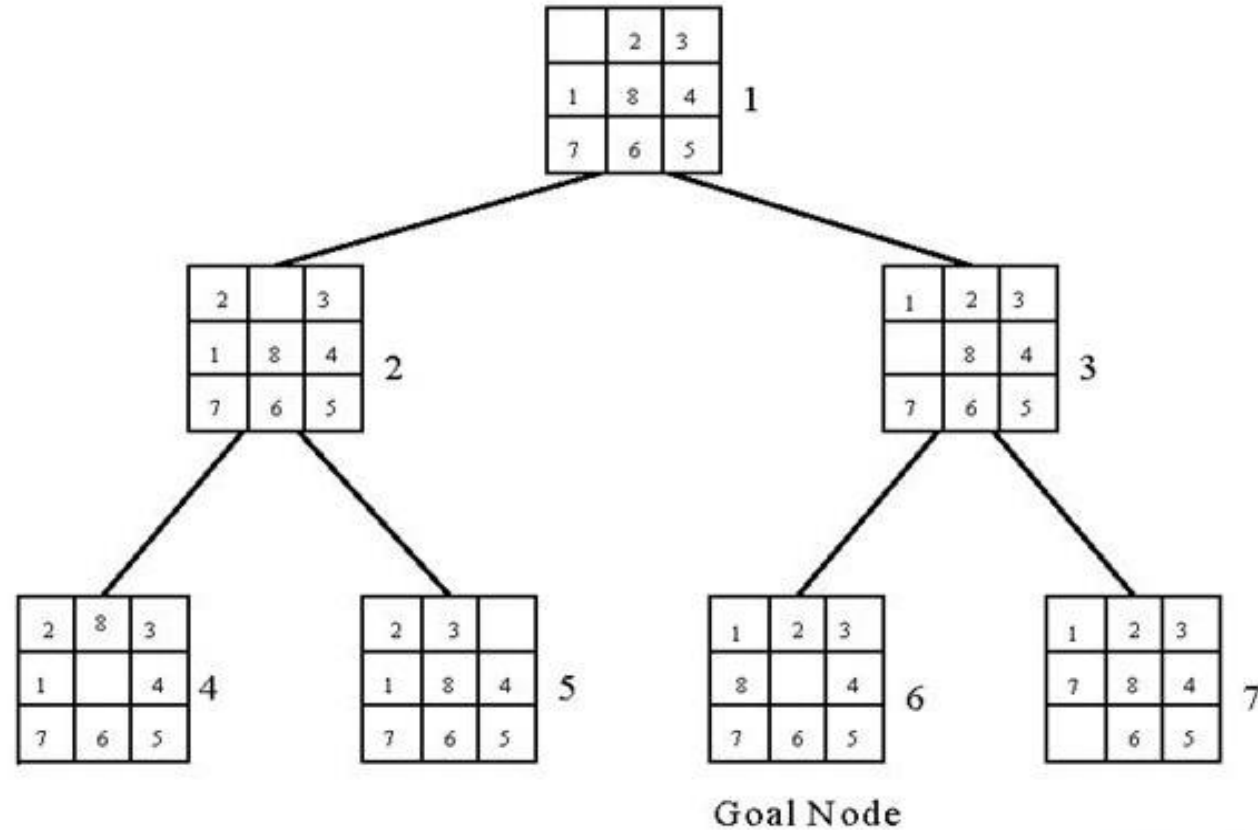
- Goal State

1	2	3
8		4
7	6	5

State Space Tree

8-puzzle problem

- Solve
 - BFS
 - DFS
 - DLS
 - IDS



Exercise

